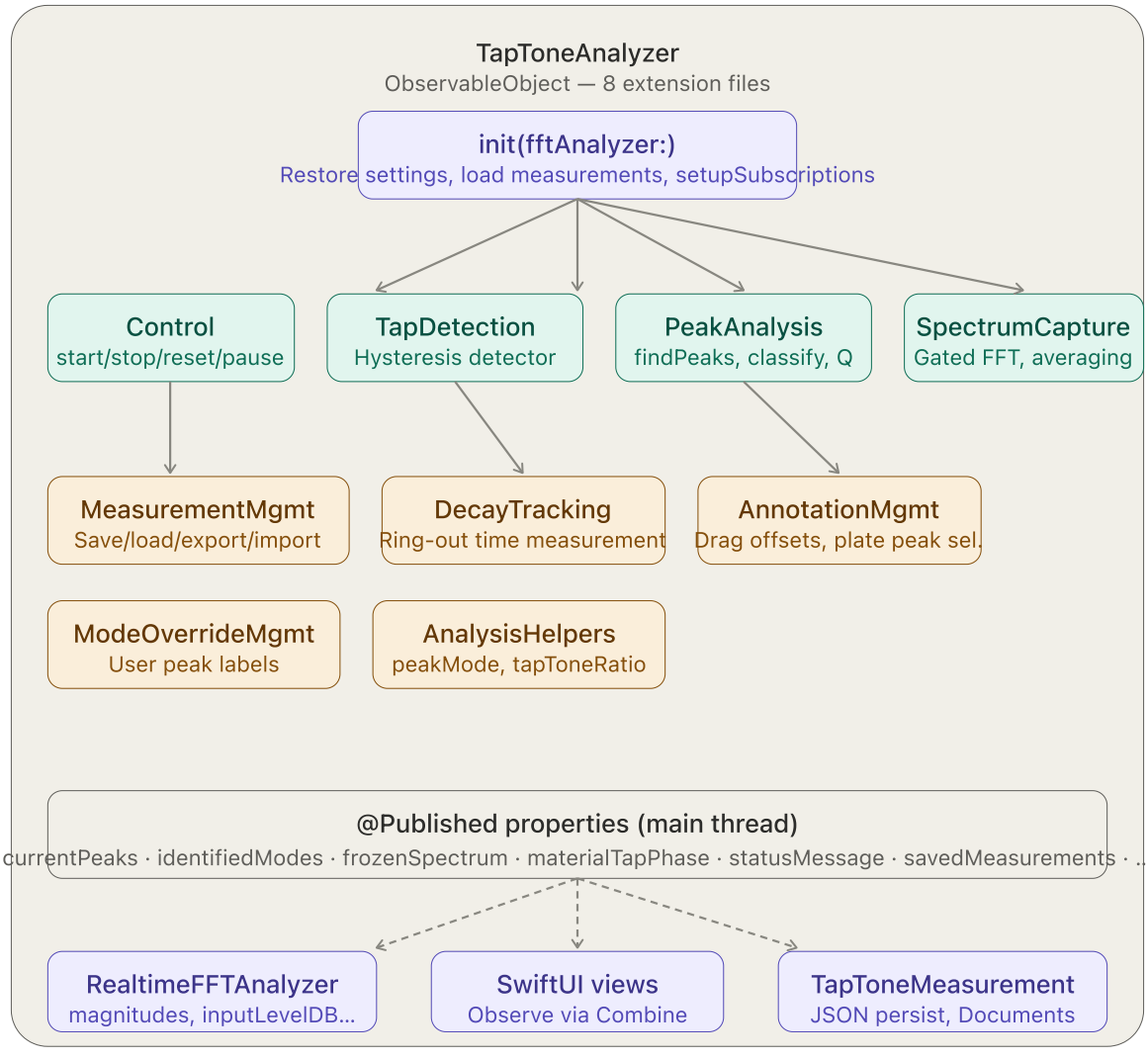
GuitarTap — TapToneAnalyzer Architecture

Generated from source analysis · 8 extension files

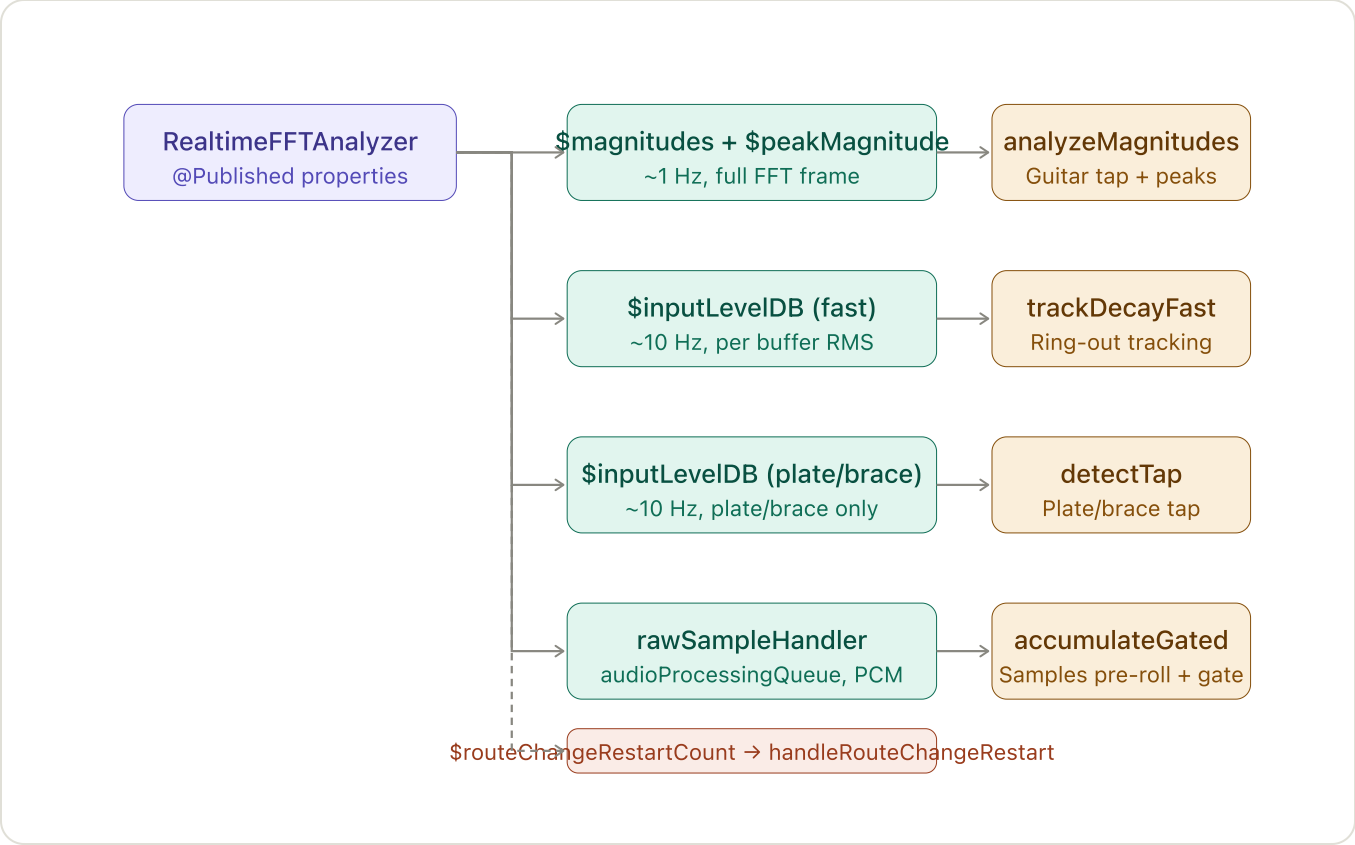
1. Full class architecture

`TapToneAnalyzer` sits between `RealtimeFFTAnalyzer` (audio/FFT engine) and SwiftUI views. It owns the complete measurement lifecycle: waiting for a tap, capturing the ring-out, detecting peaks, classifying modes, and persisting results. The base file holds all stored properties and wires up Combine subscriptions in `init`.



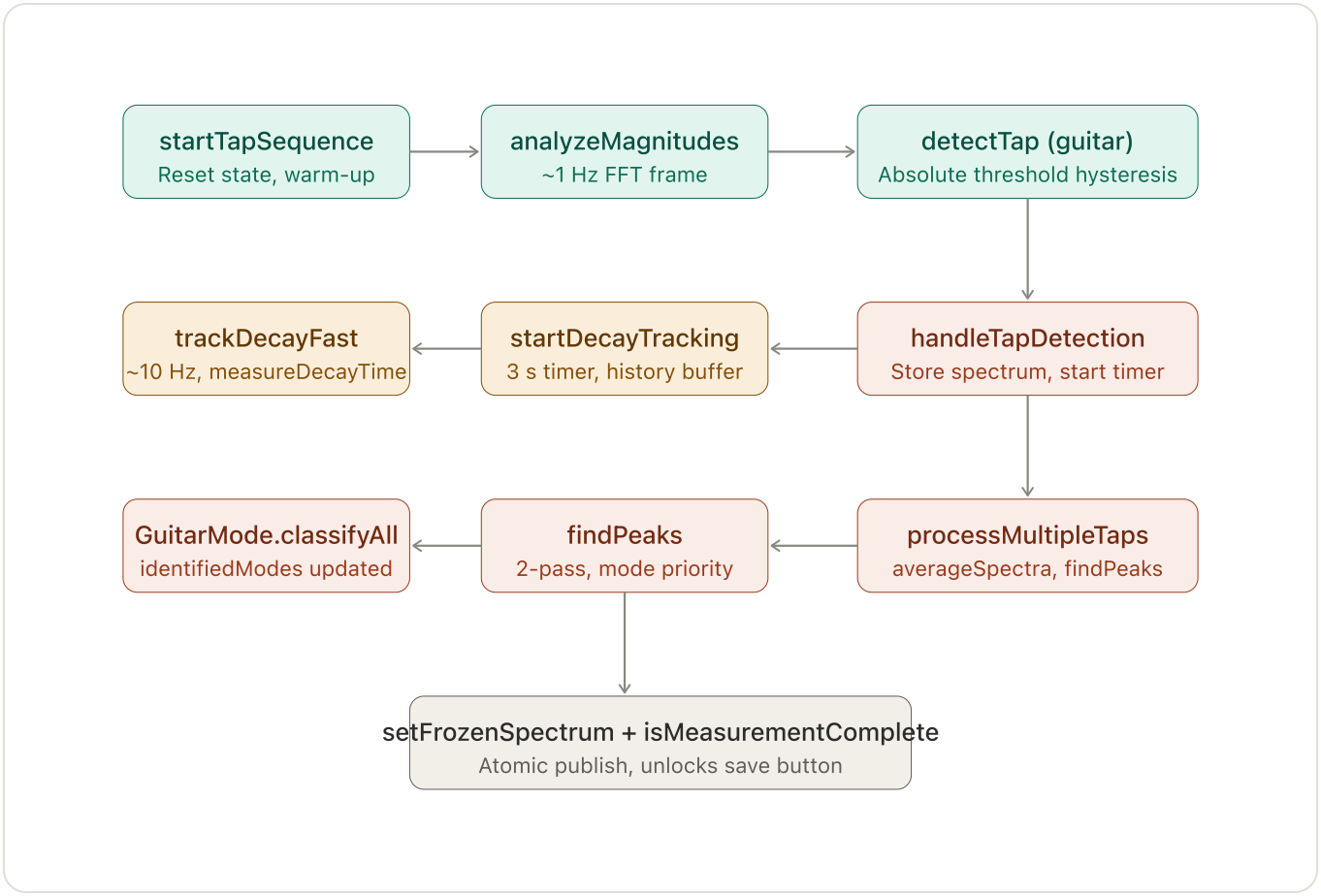
2. Combine subscriptions (setupSubscriptions)

Four subscriptions wire the FFT engine to the analyzer. The *slow path* uses the full FFT frame (~1 Hz) for guitar-mode tap detection and peak analysis. The *fast path* uses `inputLevelDB` (~10 Hz) for decay tracking and plate/brace tap detection — the long-window FFT dilutes tap transients by ~15 dB, making it unsuitable for short-impact detection in plate/brace mode.



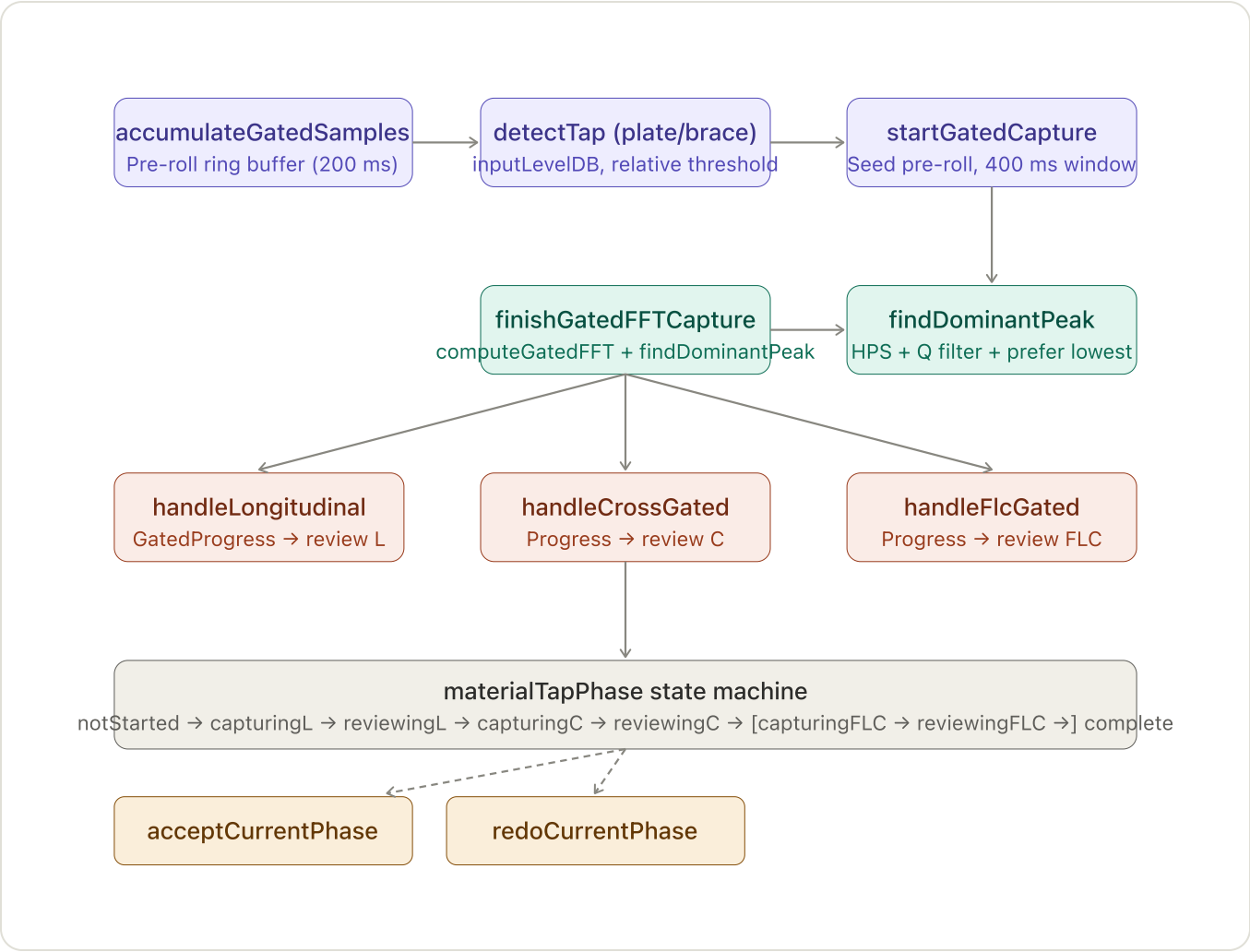
3. Guitar mode measurement flow

Guitar mode uses the continuous FFT path for tap detection. `detectTap` runs an absolute-threshold hysteresis state machine. On a rising edge, the current spectrum is stored, decay tracking starts, and a capture-window timer fires after 200 ms. When all taps are collected, `processMultipleTaps` power-averages the spectra and runs `findPeaks` on the result.



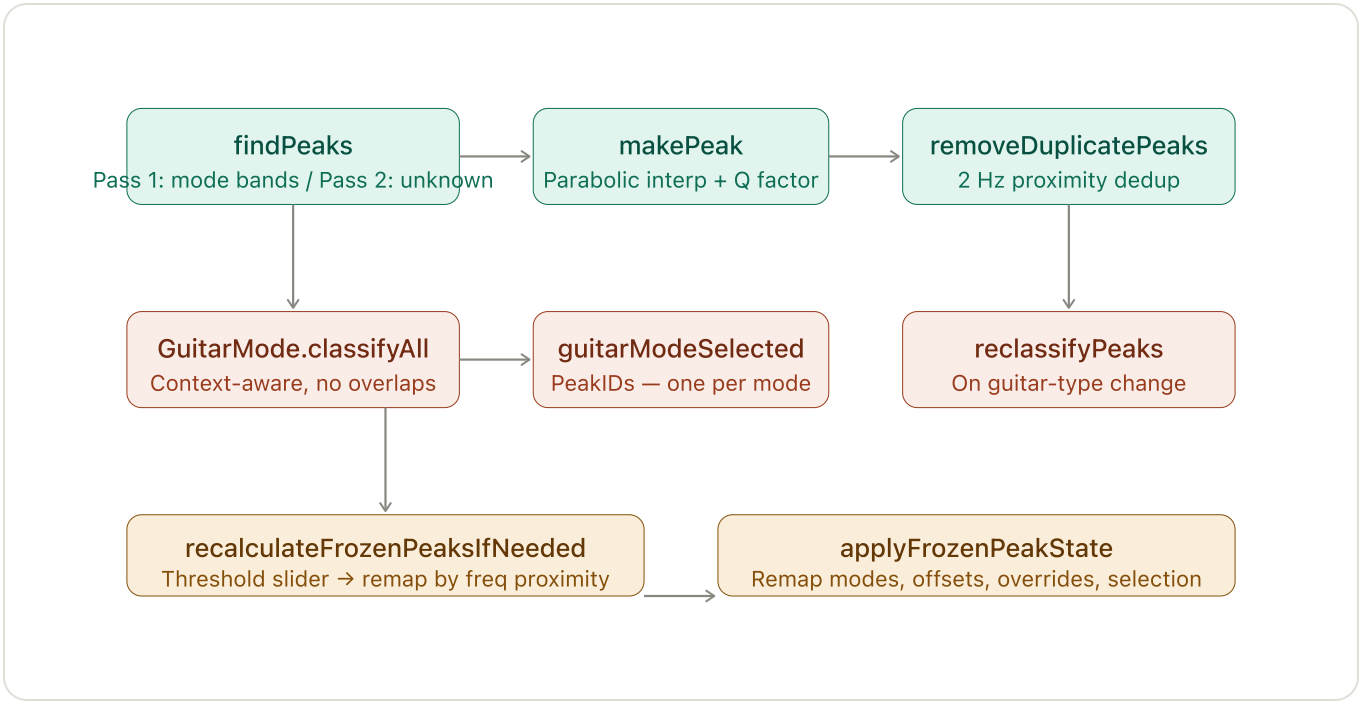
4. Plate / brace gated FFT flow

Plate and brace measurements bypass the continuous FFT path entirely. A pre-roll ring buffer (200 ms) continuously holds the most recent audio. When a tap is detected via `inputLevelDB`, `startGatedCapture` seeds the accumulator with the pre-roll so the tap attack is not missed, then accumulates a 400 ms window. `finishGatedFFTCapture` runs a dedicated FFT and routes results to the phase handler.



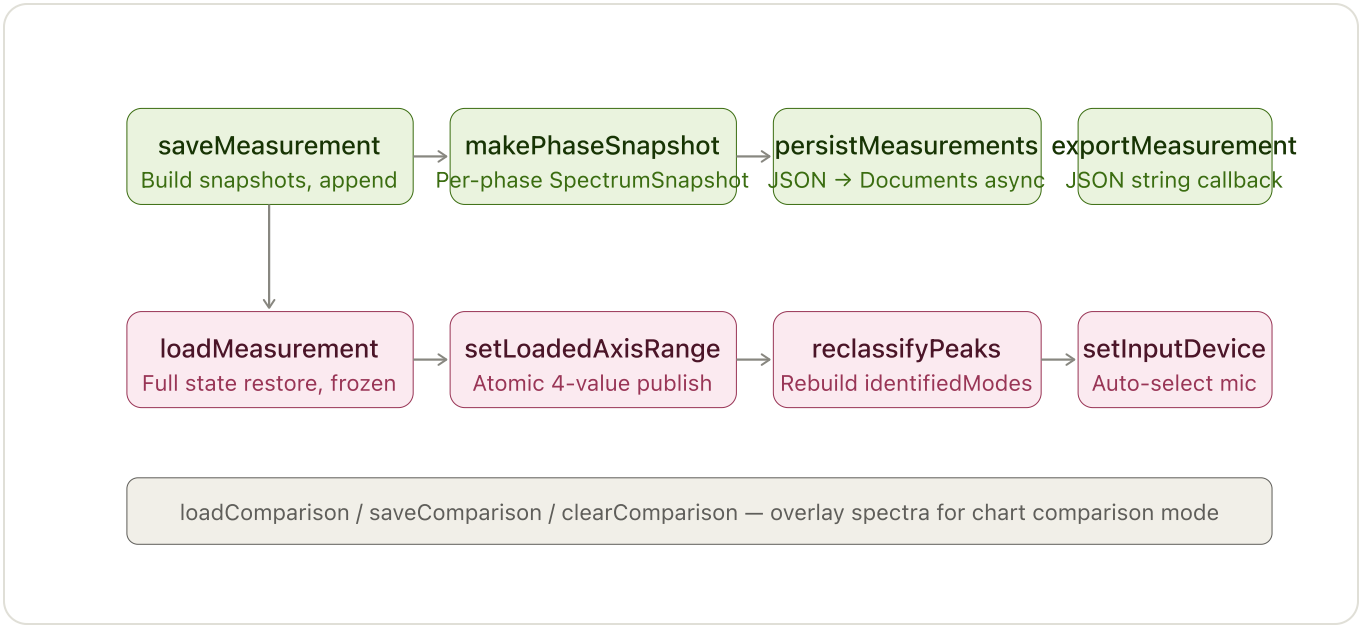
5. Peak analysis pipeline (PeakAnalysis)

`findPeaks` uses a two-pass strategy: Pass 1 scans each known guitar mode's frequency band and guarantees one slot per mode. Pass 2 scans inter-mode regions for unknown peaks. All peaks are refined with parabolic interpolation and Q-factor calculation. `recalculateFrozenPeaksIfNeeded` re-runs this on the frozen spectrum when the threshold slider moves, remapping UUIDs by frequency proximity to preserve annotation offsets and mode overrides.



6. Measurement save / load flow

`saveMeasurement` builds per-phase `SpectrumSnapshot` objects internally (not by the view), then appends a `TapToneMeasurement` to `savedMeasurements` and writes the JSON array to Documents asynchronously. `loadMeasurement` is the most complex method — it restores peaks, spectra, axis ranges, analysis settings, annotation offsets, mode overrides, and microphone selection, all suppressing `recalculateFrozenPeaksIfNeeded` during the load via the `isLoadingMeasurement` guard.



Key architecture notes

Three-layer peak selection (plate / brace)

Peak selection resolves with strict priority: `userSelectedXxxPeakID` (user tapped a row) beats `selectedXxxPeak` (stored when phase completes) beats `autoSelectedXxxPeakID` (HPS analysis result). Views and material-property calculations read `effectiveLongitudinalPeakID` / `effectiveCrossPeakID` / `effectiveFlcPeakID` which implement this resolution.

Tap detection mode split

Mode	Signal	Threshold	Reason
Guitar	FFT `peakMagnitude` (~1 Hz)	Absolute dBFS	Full spectrum available for analysis at tap time
Plate / Brace	`inputLevelDB` (~10 Hz RMS)	Relative: noise floor + headroom	Long FFT window dilutes tap energy by ~15 dB

Atomic publishing guards

- `setFrozenSpectrum(frequencies:magnitudes:)` — prevents transient length mismatch that causes SwiftUI Charts vertical-line artifacts
- `setLoadedAxisRange(minFreq:maxFreq:minDB:maxDB:)` — same issue, four axis bounds as one notification
- `isLoadingMeasurement` — suppresses `recalculateFrozenPeaksIfNeeded` during load so saved peaks are not overwritten
- `isApplyingDeviceSwitch` — prevents `selectedInputDevice.didSet` from triggering a second engine restart
- `isLoadingDeviceCalibration` — prevents `activeCalibration.didSet` from re-saving the device mapping during auto-load

Warm-up and hysteresis

The first 0.5 s after `start()` or `resumeTapDetection()` suppresses all detection. On the first frame after warm-up, `justExitedWarmup` re-anchors the noise floor EMA and `isAboveThreshold` to the actual signal level without firing a spurious tap event. The hysteresis state machine uses a wider falling threshold (`tapDetectionThreshold − hysteresisMargin`) to prevent re-triggering during ring-out.

Pre-roll ring buffer

The audio tap fires on `audioProcessingQueue` ; by the time `detectTap` fires on the main thread, several audio buffers carrying the tap attack have already passed. A 200 ms circular pre-roll buffer continuously holds the most recent raw PCM. `startGatedCapture` seeds the accumulation buffer with the pre-roll so the tap transient is always captured.

Measurement persistence

All measurements are stored as a single self-contained JSON array at `Documents/saved_measurements.json`. Encoding / decoding runs on a background queue. Under XCTest the path redirects to a temporary directory. On import, microphone UIDs are matched against available devices — with a fallback to name matching for measurements from the Python companion app, which stores a `Name:SampleRate` fingerprint rather than a CoreAudio UID.

Generated by Claude from GuitarTap source analysis · TapToneAnalyzer.swift + 7 extensions